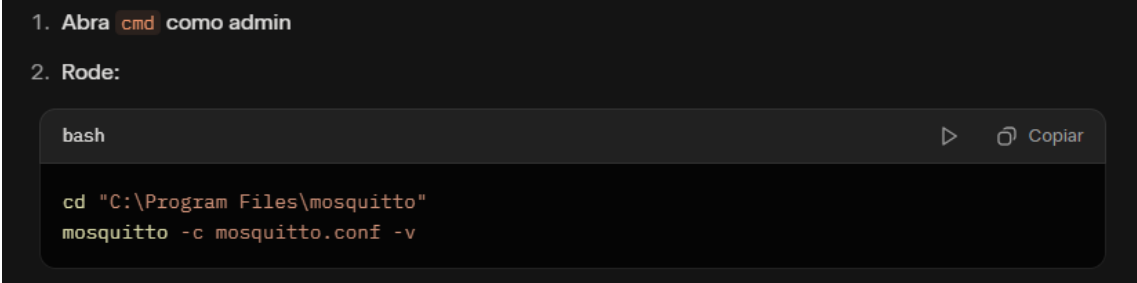


Passo a Passo – Software

Inicialização do “Mosquitto” e do “Nod-red”

-Mosquitto (broker MQTT de código aberto que funciona como um intermediário para comunicação entre dispositivos em uma rede)

- Iniciar um cmd como administrador e escrever o comando:



```
1. Abra cmd como admin
2. Rode:

bash

cd "C:\Program Files\mosquitto"
mosquitto -c mosquitto.conf -v
```

Inicializar o Nod-red (ferramenta de programação visual de baixo código (low-code) que permite criar fluxos de trabalho conectando blocos (nodes) em uma interface de navegador)

- Iniciar um cmd como administrador e escrever o comando:

```
node-red

Microsoft Windows [versão 10.0.26100.7019]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\dell 3400>node-red
13 Nov 23:52:44 - [info]

Welcome to Node-RED
=====

13 Nov 23:52:44 - [info] Node-RED version: v4.1.1
13 Nov 23:52:44 - [info] Node.js version: v22.19.0
13 Nov 23:52:44 - [info] Windows_NT 10.0.26100 x64 LE
13 Nov 23:52:44 - [info] Loading palette nodes
13 Nov 23:52:46 - [info] Settings file : C:\Users\dell 3400\.node-red\settings.js
13 Nov 23:52:46 - [info] Context store : 'default' [module=memory]
13 Nov 23:52:46 - [info] User directory : \Users\dell 3400\.node-red
13 Nov 23:52:46 - [warn] Projects disabled : editorTheme.projects.enabled=false
13 Nov 23:52:46 - [info] Flows file : \Users\dell 3400\.node-red\flows.json
13 Nov 23:52:46 - [info] Server now running at http://127.0.0.1:1880/
13 Nov 23:52:46 - [warn]

-----
Your flow credentials file is encrypted using a system-generated key.

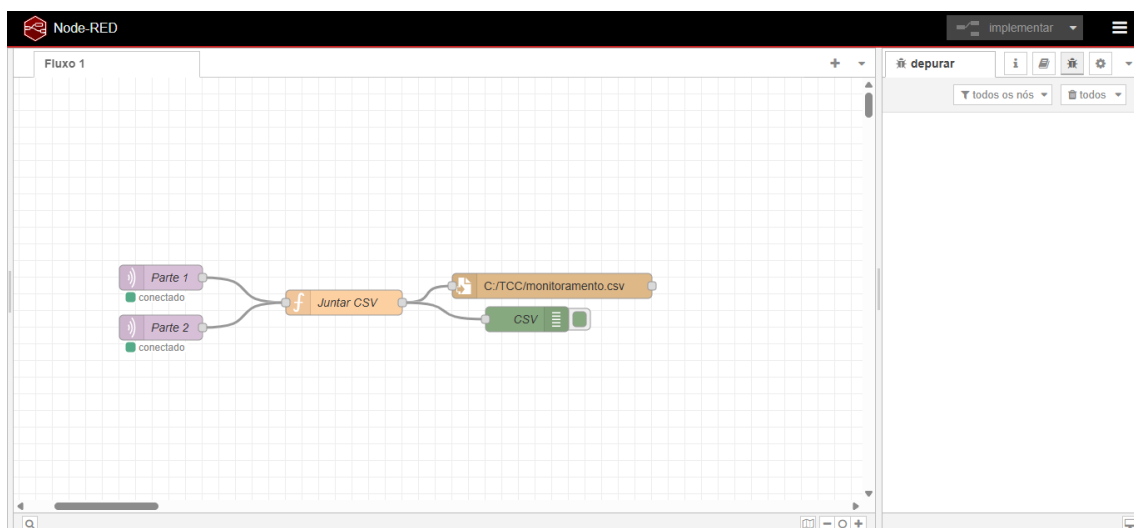
If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
-----

13 Nov 23:52:46 - [info] Starting flows
13 Nov 23:52:46 - [info] Started flows
13 Nov 23:52:46 - [info] [mqtt-broker:Mosquitto Local] Connection failed to broker: nodered_tcc@mqtt://localhost:1883
```

Colar o servido que esta rodando o nod-red , no caso da imagem é o <http://127.0.0.1:1880/> .

Abrir no Chrome :



Esse é o fluxo , a aba da direita e o debug as informações que estão sendo enviadas ao computador são mostradas lá.

Mqtt parte 1:

Editar mqtt in nó

Deletar

Cancelar

Feito

Propriedades

Servidor

Mosquitto Local

Ação

Assinar um tópico único

Tópico

Tcc/energia/parte1

QoS

0

Saída

um objeto JSON analisado sintaticamente

Nome

Parte 1

Mqtt parte 2:

Editar mqtt in nó

Deletar

Cancelar

Feito

Propriedades

Servidor

Mosquitto Local

Ação

Assinar um tópico único

Tópico

Tcc/energia/parte2

QoS

0

Saída

um objeto JSON analisado sintaticamente

Nome

Parte 2

Brokker Mqtt:

Editar mqtt in nó > **Editar mqtt-broker nó**

Excluir

Cancelar

Atualizar

Propriedades

Nome

Mosquitto Local

Conexão

Segurança

Mensagens

Servidor

localhost

Porta

1883

☒ Conectar automaticamente

☐ Usar TLS

Protocolo

MQTT V3.1.1

▼

ID do cliente

nodered_tcc

Mantenha-se vivo

60

Sessão

☒ Usar sessão limpa

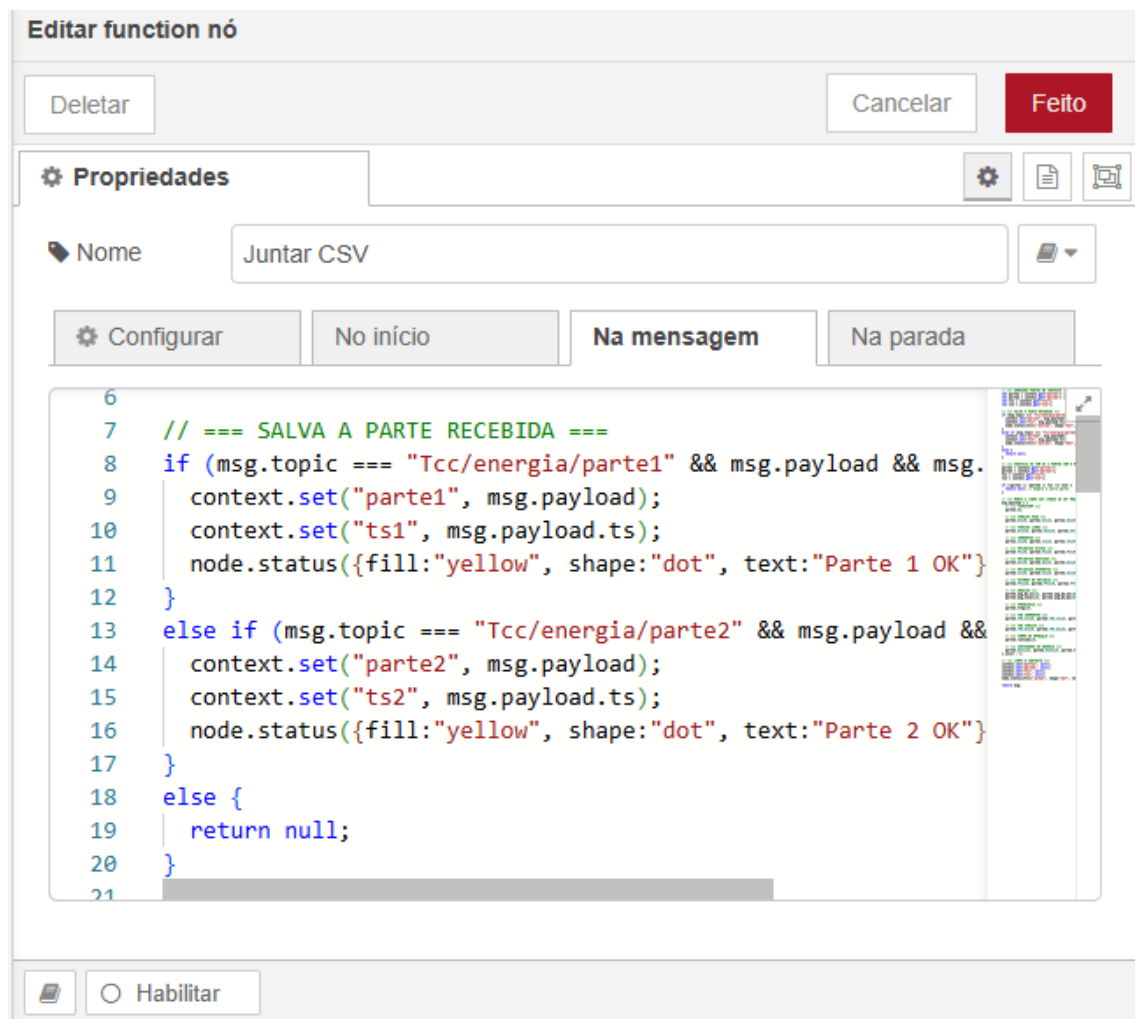
☐ Habilitar

2

Em todos os fluxos

▼

Function:



Código do Function:

```
// == ARMAZENA PARTES NO CONTEXTO ==
let parte1 = context.get("parte1") || null;
let parte2 = context.get("parte2") || null;
let ts1 = context.get("ts1");
let ts2 = context.get("ts2");

// == SALVA A PARTE RECEBIDA ==
if (msg.topic === "Tcc/energia/parte1" && msg.payload && msg.payload.ts) {
  context.set("parte1", msg.payload);
  context.set("ts1", msg.payload.ts);
  node.status({fill:"yellow", shape:"dot", text:"Parte 1 OK"});
}
else if (msg.topic === "Tcc/energia/parte2" && msg.payload &&
msg.payload.ts) {
  context.set("parte2", msg.payload);
}
```

```

    context.set("ts2", msg.payload.ts);
    node.status({fill:"yellow", shape:"dot", text:"Parte 2 OK"});
}
else {
    return null;
}

// === VERIFICA SE TEM AS 2 PARTES COM O MESMO TIMESTAMP ===
parte1 = context.get("parte1");
parte2 = context.get("parte2");
ts1 = context.get("ts1");
ts2 = context.get("ts2");

if (!parte1 || !parte2 || ts1 !== ts2) {
    return null; // Espera a outra parte
}

// === MONTA A LINHA CSV (TODAS AS 40+ MEDIÇÕES) ===
msg.payload = [
    // === TIMESTAMP ===
    parte1.ts,

    // === TENSÕES FASE ===
    parte1.U1||0, parte1.U2||0, parte1.U3||0,

    // === TENSÕES LINHA ===
    parte1.U12||0, parte1.U23||0, parte1.U31||0,

    // === CORRENTES ===
    parte1.I1||0, parte1.I2||0, parte1.I3||0, parte1.Iavg||0,

    // === POTÊNCIAS ATIVAS ===
    parte2.P1||0, parte2.P2||0, parte2.P3||0, parte2.Pt||0,

    // === POTÊNCIAS REATIVAS ===
    parte2.Q1||0, parte2.Q2||0, parte2.Q3||0, parte2.Qt||0,

    // === POTÊNCIAS APARENTES ===
    parte2.S1||0, parte2.S2||0, parte2.S3||0, parte2.St||0,

    // === FATORES DE POTÊNCIA ===
    parte2.PF1||0, parte2.PF2||0, parte2.PF3||0, parte2.PFt||0,

    // === ÂNGULOS ===
    parte2.ang_U1_I1||0, parte2.ang_U2_I2||0, parte2.ang_U3_I3||0,
    parte2.ang_total||0, parte2.ang_U1_U2||0, parte2.ang_U2_U3||0,
    parte2.ang_U3_U1||0,

    // === FREQUÊNCIA ===

```

```
parte2.freq||0,

// === THD CORRENTES ===
parte2.THD_I1||0, parte2.THD_I2||0, parte2.THD_I3||0,

// === THD TENSÕES ===
parte2.THD_U1||0, parte2.THD_U2||0, parte2.THD_U3||0,

// === TEMPO DE OPERAÇÃO ===
parte2.runtime||0,

// === CONTADORES DE ENERGIA ===
parte2.Kwh1||0, parte2.Kwh2||0, parte2.Kvarh3||0, parte2.Kvarh4||0
].join(',');

// === LIMPA O CONTEXTO ===
context.set("parte1", null);
context.set("parte2", null);
context.set("ts1", null);
context.set("ts2", null);
node.status({fill:"green", shape:"dot", text:"CSV GERADO (40+ campos)"});

return msg;
```

Nó de escrever no csv:

Editar escrever arquivo nó

Deletar

Cancelar

Feito

 **Propriedades**



 Nome do arquivo

▼ caminho

C:/TCC/monitoramento.csv

 Ação

adicionar ao arquivo

▼

☒ Adicionar nova linha (\n) a cada carga útil?

☒ Criar diretório se não existir?

 Codificação

▼

 Nome

Dica: O nome do arquivo deve ser um caminho absoluto, caso contrário, será relativo ao diretório de trabalho do processo Node-RED.



☐ Habilitar

Debug:

Editar debug nó

Deletar

Cancelar

Feito

Propriedades

Saída

▼ msg. payload

Para

☒ janela de depuração

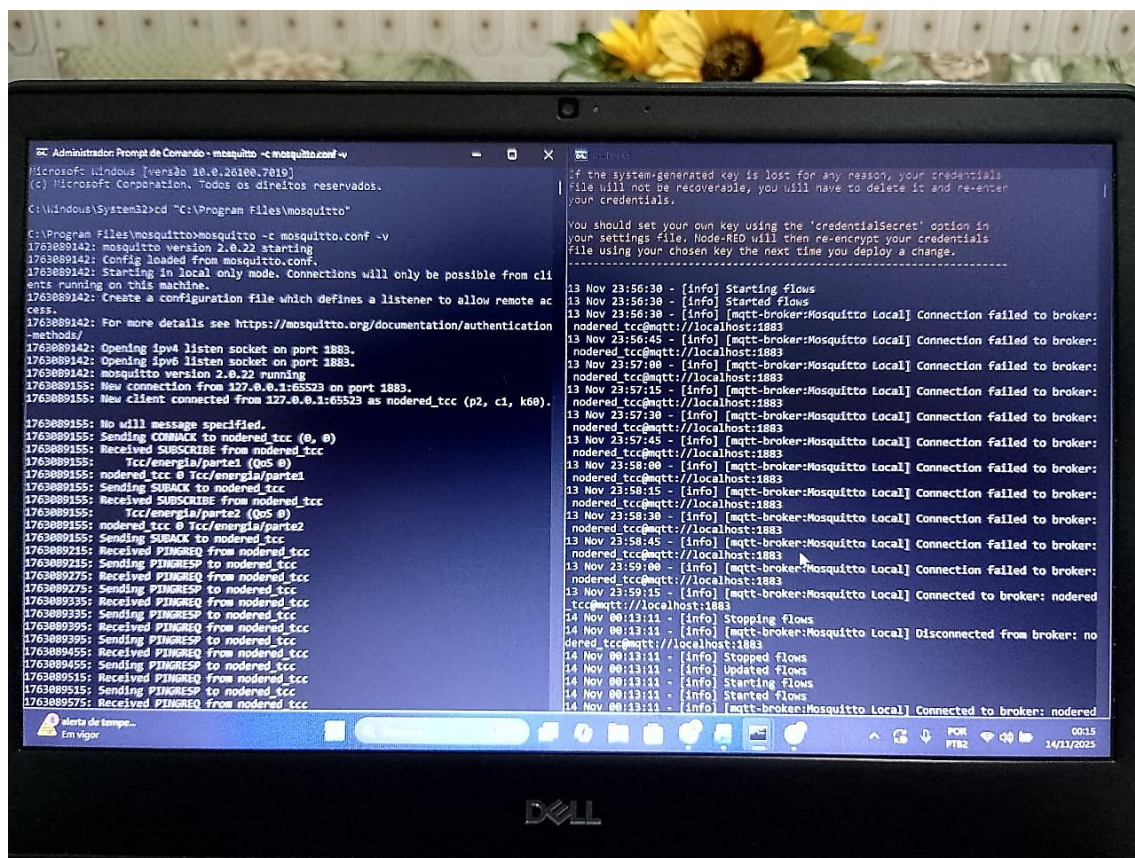
☐ console do sistema

☐ estado do nó (32 caracteres)

Nome

CSV

Os cmds tem que estar assim:



```
Administrador: Prompt de Comando - mosquitto -c mosquitto.conf -v
Microsoft Windows [versão 10.0.26100.7819]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Windows\System32>cd "C:\Program Files\mosquitto"

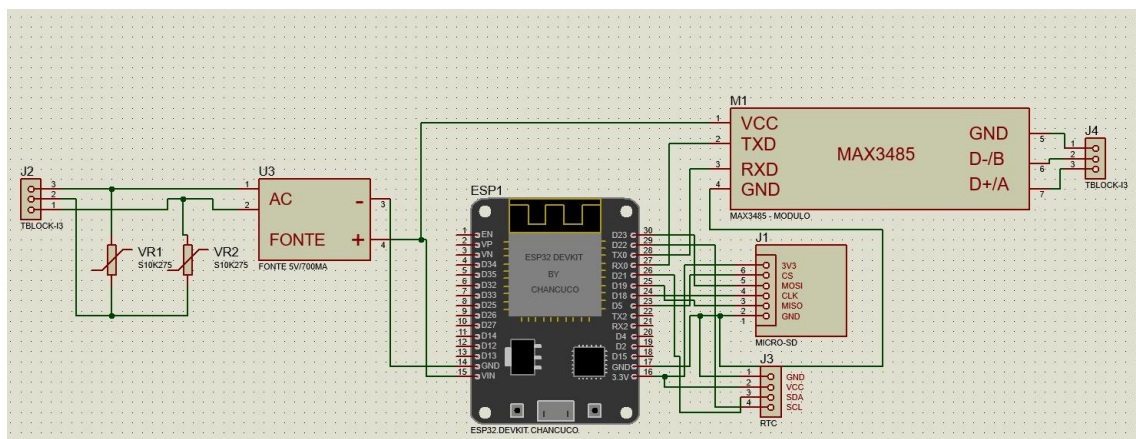
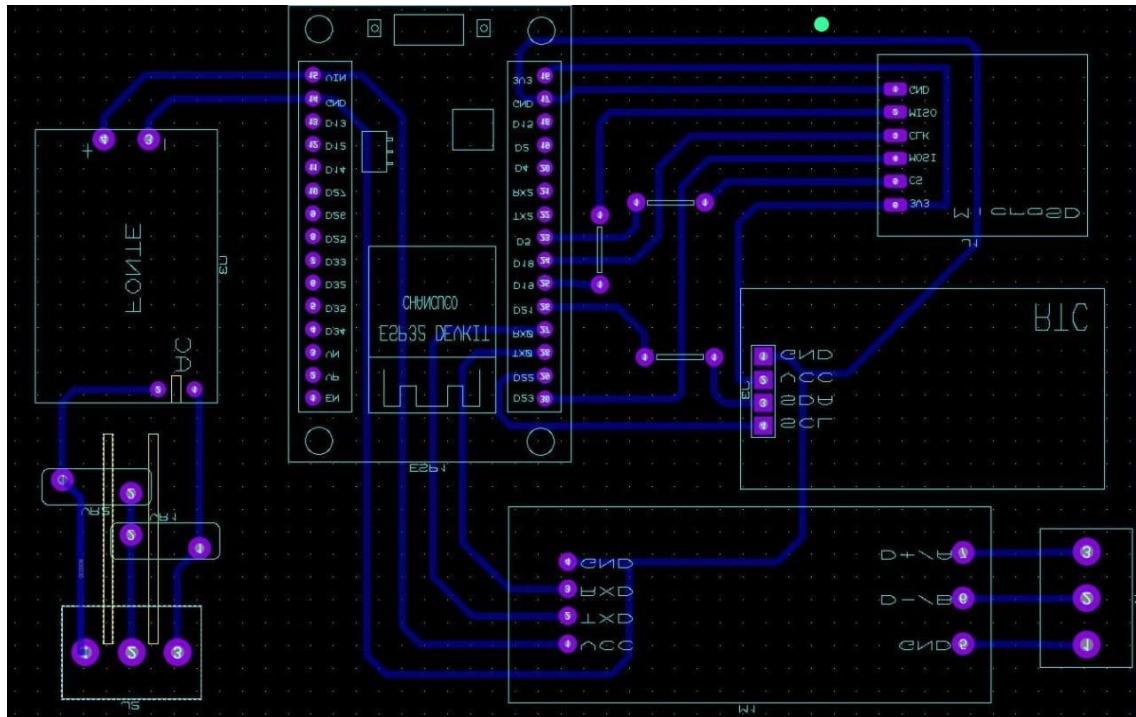
C:\Program Files\mosquitto>mosquitto -c mosquitto.conf -v
1763089142: mosquitto version 2.0.22 starting
1763089142: Config loaded from mosquitto.conf:
1763089142: Starting in local only mode. Connections will only be possible from cli
ents running on this machine.
1763089142: Create a configuration file which defines a listener to allow remote ac
cess.
1763089142: For more details see https://mosquitto.org/documentation/authentication
-methods/
1763089142: Opening ipv4 listen socket on port 1883.
1763089142: Opening ipv6 listen socket on port 1883.
1763089155: mosquitto version 2.0.22 running
1763089155: New connection from 127.0.0.1:65523 on port 1883.
1763089155: New client connected from 127.0.0.1:65523 as nodered_tcc (p2, c1, k60).

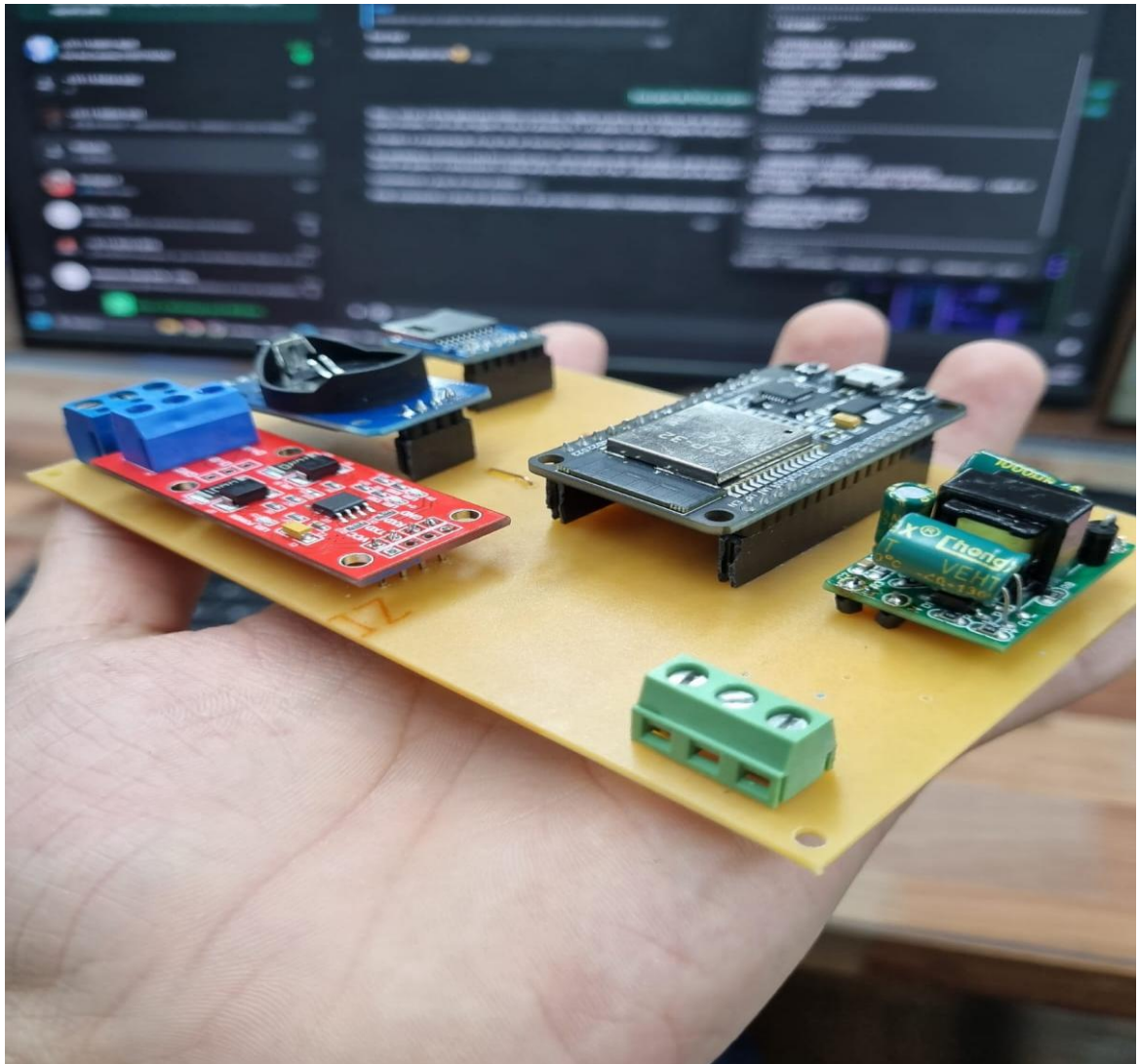
1763089155: No will message specified.
1763089155: Sending CONNACK to nodered_tcc (0, 0)
1763089155: Received SUBSCRIBE from nodered_tcc
1763089155:   tcc/energia/part1 (qos 0)
1763089155: nodered_tcc 0 tcc/energia/part1
1763089155: Sending SUBACK to nodered_tcc
1763089155: Received SUBSCRIBE from nodered_tcc
1763089155:   tcc/energia/part2 (qos 0)
1763089155: nodered_tcc 0 tcc/energia/part2
1763089155: Sending SUBACK to nodered_tcc
1763089215: Received PINGREQ from nodered_tcc
1763089215: Sending PINGRESP to nodered_tcc
1763089275: Received PINGREQ from nodered_tcc
1763089275: Sending PINGRESP to nodered_tcc
1763089335: Received PINGREQ from nodered_tcc
1763089335: Sending PINGRESP to nodered_tcc
1763089395: Received PINGREQ from nodered_tcc
1763089395: Sending PINGRESP to nodered_tcc
1763089455: Received PINGREQ from nodered_tcc
1763089455: Sending PINGRESP to nodered_tcc
1763089515: Received PINGREQ from nodered_tcc
1763089515: Sending PINGRESP to nodered_tcc
1763089575: Received PINGREQ from nodered_tcc

13 Nov 23:56:30 - [info] Starting flows
13 Nov 23:56:30 - [info] Started flows
13 Nov 23:56:30 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:56:45 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:57:00 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:57:15 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:57:30 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:57:45 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:58:00 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:58:15 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:58:30 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:58:45 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:59:00 - [info] [mqtt-broker:mosquitto local] Connection failed to broker:
nodered_tcc@mqtt://localhost:1883
13 Nov 23:59:15 - [info] [mqtt-broker:mosquitto local] Connected to broker: nodered
_tcc@mqtt://localhost:1883
14 Nov 00:13:11 - [info] Stopping flows
14 Nov 00:13:11 - [info] [mqtt-broker:mosquitto local] Disconnected from broker: no
dered_tcc@mqtt://localhost:1883
14 Nov 00:13:11 - [info] Stopped flows
14 Nov 00:13:11 - [info] Updated flows
14 Nov 00:13:11 - [info] Starting flows
14 Nov 00:13:11 - [info] Started flows
14 Nov 00:13:11 - [info] [mqtt-broker:mosquitto local] Connected to broker: nodered
_tcc@mqtt://localhost:1883

aviso de tempo...
Em vigor
```

A placa montada com o seguinte circuito:





Código para carregar no esp:

```
/*
*****
* SISTEMA DE MONITORAMENTO DE ENERGIA ELÉTRICA
* Projeto TCC - SENAI Automação
* Data: 12 de Novembro de 2025
* Autor: [Rodrigo Pericinotto]
* Descrição: ESP32 lê medidor 7M via Modbus RTU, sincroniza RTC com NTP,
*             salva dados no SD (CSV por grandeza), e envia 40+ medições
*             via MQTT em 2 partes JSON (parte1: tensões/correntes,
*             parte2: potências/ângulos/THD/contadores).
*****
*****/

#include <WiFi.h>           // Biblioteca para conexão WiFi
#include "RTCLib.h"         // Controle do módulo RTC DS3231
#include "time.h"           // Funções de tempo (NTP)
#include <ModbusMaster.h>   // Comunicação Modbus RTU
```

```

#include <SD.h> // Acesso ao cartão SD
#include <SPI.h> // Comunicação SPI (SD)
#include <PubSubClient.h> // Cliente MQTT
#include <ArduinoJson.h> // Serialização JSON

#define MQTT_MAX_PACKET_SIZE 1024 // Tamanho máximo do pacote MQTT

const char* ssid = "SENAI_AUTOMACAO"; // Nome da rede WiFi
const char* password = "senai118"; // Senha da rede WiFi
const char* mqtt_server = "192.168.18.100"; // IP do broker MQTT
(Mosquitto)
const int mqtt_port = 1883; // Porta MQTT padrão

RTC_DS3231 rtc; // Objeto RTC
#define RXD2 16 // Pino RX da Serial2
(Modbus)
#define TXD2 17 // Pino TX da Serial2
(Modbus)
constexpr uint8_t MODBUS_7M_ADDRESS = 1; // Endereço Modbus do
medidor
ModbusMaster node; // Objeto Modbus
#define SD_CS 5 // Pino CS do cartão SD

WiFiClient espClient; // Cliente WiFi
PubSubClient client(espClient); // Cliente MQTT

const char* ntpServer = "pool.ntp.org"; // Servidor NTP
const long gmtOffset_sec = -3 * 3600; // Fuso horário: UTC-3
(Brasil)
const int daylightOffset_sec = 0; // Sem horário de verão

// ===== AJUSTAR HORA VIA NTP =====
void ajustarViaNTP() {
    Serial.println("Ajustando hora via NTP...");
    configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
    struct tm timeinfo;
    int tentativas = 0;
    while (!getLocalTime(&timeinfo) && tentativas < 10) {
        delay(1000);
        tentativas++;
    }
    if (getLocalTime(&timeinfo)) {
        rtc.adjust(DateTime(timeinfo.tm_year + 1900, timeinfo.tm_mon + 1,
timeinfo.tm_mday,
timeinfo.tm_hour, timeinfo.tm_min,
timeinfo.tm_sec));
        Serial.println("RTC ajustado via NTP!");
    }
}

```

```

// ===== LER FLOAT IEEE754 DE 2 REGISTRADORES =====
float readFloatIEEE(uint16_t reg) {
    uint8_t result = node.readInputRegisters(reg, 2);
    if (result == node.ku8MBSuccess) {
        uint16_t w0 = node.getResponseBuffer(0);
        uint16_t w1 = node.getResponseBuffer(1);
        uint32_t rawHL = ((uint32_t)w0 << 16) | w1;
        uint32_t rawLH = ((uint32_t)w1 << 16) | w0;
        float fHL, fLH;
        memcpy(&fHL, &rawHL, 4);
        memcpy(&fLH, &rawLH, 4);
        if (fHL > 0.1 && fHL < 80000) return fHL;
        if (fLH > 0.1 && fLH < 80000) return fLH;
    }
    return 0;
}

// ===== GRAVAR NO SD (CSV SEPARADOS POR GRANDEZA) =====
void salvarNoSD(const char* label, float value, const char* filename) {
    DateTime now = rtc.now();
    String dateStr = String(now.year()) + "-" +
                     (now.month() < 10 ? "0" : "") + String(now.month()) +
    "-" +
                     (now.day() < 10 ? "0" : "") + String(now.day());
    String timeStr = (now.hour() < 10 ? "0" : "") + String(now.hour()) +
    ":" +
                     (now.minute() < 10 ? "0" : "") + String(now.minute())
    + ":" +
                     (now.second() < 10 ? "0" : "") + String(now.second());

    File file = SD.open(filename, FILE_APPEND);
    if (file) {
        if (file.size() == 0) {
            file.println("Grandeza,Valor,Data,Hora");
        }
        file.print(label); file.print(",");
        file.print(value, 2); file.print(",");
        file.print(dateStr); file.print(",");
        file.println(timeStr);
        file.close();
    }
}

// ===== RECONECTAR WIFI =====
void reconectarWiFi() {
    if (WiFi.status() == WL_CONNECTED) return;
    WiFi.disconnect();
    WiFi.begin(ssid, password);
}

```

```

    int tentativas = 0;
    while (WiFi.status() != WL_CONNECTED && tentativas < 40) {
        delay(500); Serial.print("."); tentativas++;
    }
    Serial.println(WiFi.status() == WL_CONNECTED ? "\nWiFi OK: " +
    WiFi.localIP().toString() : "\nWiFi FALHOU");
}

// ===== RECONECTAR MQTT =====
void reconectarMQTT() {
    if (WiFi.status() != WL_CONNECTED) return;
    int tentativas = 0;
    while (!client.connected() && tentativas < 5) {
        if (client.connect("ESP32_TCC")) {
            Serial.println("MQTT CONECTADO!");
            return;
        }
        delay(3000); tentativas++;
    }
}

// ===== ENVIA PARTE 1 (TENSÕES + CORRENTES) =====
void enviarParte1() {
    StaticJsonDocument<1024> doc;
    DateTime now = rtc.now();
    char ts[20];
    sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d", now.year(), now.month(),
    now.day(), now.hour(), now.minute(), now.second());
    doc["ts"] = ts;

    doc["U1"] = readFloatIEEE(32500 - 30001);
    doc["U2"] = readFloatIEEE(32502 - 30001);
    doc["U3"] = readFloatIEEE(32504 - 30001);
    doc["U12"] = readFloatIEEE(32508 - 30001);
    doc["U23"] = readFloatIEEE(32510 - 30001);
    doc["U31"] = readFloatIEEE(32512 - 30001);
    doc["I1"] = readFloatIEEE(32516 - 30001);
    doc["I2"] = readFloatIEEE(32518 - 30001);
    doc["I3"] = readFloatIEEE(32520 - 30001);
    doc["Iavg"] = readFloatIEEE(32528 - 30001);

    char buffer[1024];
    size_t len = serializeJson(doc, buffer);
    client.publish("Tcc/energia/parte1", buffer, len);
}

// ===== ENVIA PARTE 2 (POTÊNCIAS, ÂNGULOS, THD, CONTADORES) =====
void enviarParte2() {
    StaticJsonDocument<1024> doc;

```

```

DateTime now = rtc.now();
char ts[20];
sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d", now.year(), now.month(),
now.day(), now.hour(), now.minute(), now.second());
doc["ts"] = ts;

doc["P1"] = readFloatIEEE(32530 - 30001);
doc["P2"] = readFloatIEEE(32532 - 30001);
doc["P3"] = readFloatIEEE(32534 - 30001);
doc["Pt"] = readFloatIEEE(32536 - 30001);
doc["Q1"] = readFloatIEEE(32538 - 30001);
doc["Q2"] = readFloatIEEE(32540 - 30001);
doc["Q3"] = readFloatIEEE(32542 - 30001);
doc["Qt"] = readFloatIEEE(32544 - 30001);
doc["S1"] = readFloatIEEE(32546 - 30001);
doc["S2"] = readFloatIEEE(32548 - 30001);
doc["S3"] = readFloatIEEE(32550 - 30001);
doc["St"] = readFloatIEEE(32552 - 30001);
doc["PF1"] = readFloatIEEE(32554 - 30001);
doc["PF2"] = readFloatIEEE(32556 - 30001);
doc["PF3"] = readFloatIEEE(32558 - 30001);
doc["Pft"] = readFloatIEEE(32560 - 30001);

doc["ang_U1_I1"] = readFloatIEEE(32570 - 30001);
doc["ang_U2_I2"] = readFloatIEEE(32572 - 30001);
doc["ang_U3_I3"] = readFloatIEEE(32574 - 30001);
doc["ang_total"] = readFloatIEEE(32576 - 30001);
doc["ang_U1_U2"] = readFloatIEEE(32578 - 30001);
doc["ang_U2_U3"] = readFloatIEEE(32580 - 30001);
doc["ang_U3_U1"] = readFloatIEEE(32582 - 30001);

doc["freq"] = readFloatIEEE(32584 - 30001);

doc["THD_I1"] = readFloatIEEE(32588 - 30001);
doc["THD_I2"] = readFloatIEEE(32590 - 30001);
doc["THD_I3"] = readFloatIEEE(32592 - 30001);
doc["THD_U1"] = readFloatIEEE(32594 - 30001);
doc["THD_U2"] = readFloatIEEE(32596 - 30001);
doc["THD_U3"] = readFloatIEEE(32598 - 30001);

doc["runtime"] = readFloatIEEE(32480 - 30001);
doc["Kwh1"] = readFloatIEEE(32638 - 30001);
doc["Kwh2"] = readFloatIEEE(32640 - 30001);
doc["Kvarh3"] = readFloatIEEE(32642 - 30001);
doc["Kvarh4"] = readFloatIEEE(32644 - 30001);

char buffer[1024];
size_t len = serializeJson(doc, buffer);
client.publish("Tcc/energia/parte2", buffer, len);

```

```

}

// ===== SETUP =====
void setup() {
  Serial.begin(115200);
  Serial2.begin(38400, SERIAL_8N1, RXD2, TXD2);
  node.begin(MODBUS_7M_ADDRESS, Serial2);

  if (!rtc.begin()) { Serial.println("RTC FALHOU!"); while (1); }
  if (rtc.lostPower()) ajustarViaNTP();

  if (!SD.begin(SD_CS)) Serial.println("SD FALHOU");
  else Serial.println("SD OK");

  WiFi.begin(ssid, password);
  int tentativas = 0;
  while (WiFi.status() != WL_CONNECTED && tentativas < 40) { delay(500);
Serial.print("."); tentativas++; }
  Serial.println(WiFi.status() == WL_CONNECTED ? "\nWiFi OK: " +
WiFi.localIP().toString() : "\nWiFi FALHOU");

  if (WiFi.status() == WL_CONNECTED) ajustarViaNTP();

  if (WiFi.status() == WL_CONNECTED) {
    client.setServer(mqtt_server, mqtt_port);
    client.setBufferSize(1024);
    reconectarMQTT();
  }

  Serial.println("=== TCC PRONTO ===");
}

// ===== LOOP =====
void loop() {
  if (WiFi.status() != WL_CONNECTED) reconectarWiFi();
  if (WiFi.status() == WL_CONNECTED && !client.connected())
reconectarMQTT();
  client.loop();

  static unsigned long last = 0;
  if (millis() - last < 10000) return;
  last = millis();

  Serial.println("\nNOVA LEITURA");

  // === LEITURA + SD ===
  float U1 = readFloatIEEE(32500 - 30001); salvarNoSD("U1", U1,
"/tensao.csv");

```

```
float U2 = readFloatIEEE(32502 - 30001); salvarNoSD("U2", U2,
"/tensao.csv");
float U3 = readFloatIEEE(32504 - 30001); salvarNoSD("U3", U3,
"/tensao.csv");
float U12 = readFloatIEEE(32508 - 30001); salvarNoSD("U12", U12,
"/tensao_linha.csv");
float U23 = readFloatIEEE(32510 - 30001); salvarNoSD("U23", U23,
"/tensao_linha.csv");
float U31 = readFloatIEEE(32512 - 30001); salvarNoSD("U31", U31,
"/tensao_linha.csv");
float I1 = readFloatIEEE(32516 - 30001); salvarNoSD("I1", I1,
"/corrente.csv");
float I2 = readFloatIEEE(32518 - 30001); salvarNoSD("I2", I2,
"/corrente.csv");
float I3 = readFloatIEEE(32520 - 30001); salvarNoSD("I3", I3,
"/corrente.csv");
float Iavg = readFloatIEEE(32528 - 30001); salvarNoSD("Iavg", Iavg,
"/corrente.csv");
float P1 = readFloatIEEE(32530 - 30001); salvarNoSD("P1", P1,
"/potencia_ativa.csv");
float P2 = readFloatIEEE(32532 - 30001); salvarNoSD("P2", P2,
"/potencia_ativa.csv");
float P3 = readFloatIEEE(32534 - 30001); salvarNoSD("P3", P3,
"/potencia_ativa.csv");
float Pt = readFloatIEEE(32536 - 30001); salvarNoSD("Pt", Pt,
"/potencia_ativa.csv");
float Q1 = readFloatIEEE(32538 - 30001); salvarNoSD("Q1", Q1,
"/potencia_reativa.csv");
float Q2 = readFloatIEEE(32540 - 30001); salvarNoSD("Q2", Q2,
"/potencia_reativa.csv");
float Q3 = readFloatIEEE(32542 - 30001); salvarNoSD("Q3", Q3,
"/potencia_reativa.csv");
float Qt = readFloatIEEE(32544 - 30001); salvarNoSD("Qt", Qt,
"/potencia_reativa.csv");
float S1 = readFloatIEEE(32546 - 30001); salvarNoSD("S1", S1,
"/potencia_aparente.csv");
float S2 = readFloatIEEE(32548 - 30001); salvarNoSD("S2", S2,
"/potencia_aparente.csv");
float S3 = readFloatIEEE(32550 - 30001); salvarNoSD("S3", S3,
"/potencia_aparente.csv");
float St = readFloatIEEE(32552 - 30001); salvarNoSD("St", St,
"/potencia_aparente.csv");
float PF1 = readFloatIEEE(32554 - 30001); salvarNoSD("PF1", PF1,
"/fator_potencia.csv");
float PF2 = readFloatIEEE(32556 - 30001); salvarNoSD("PF2", PF2,
"/fator_potencia.csv");
float PF3 = readFloatIEEE(32558 - 30001); salvarNoSD("PF3", PF3,
"/fator_potencia.csv");
```

```
float PFt = readFloatIEEE(32560 - 30001); salvarNoSD("PFt", PFt,
"/fator_potencia.csv");

float ang1 = readFloatIEEE(32570 - 30001); salvarNoSD("ang_U1_I1",
ang1, "/angulos.csv");
float ang2 = readFloatIEEE(32572 - 30001); salvarNoSD("ang_U2_I2",
ang2, "/angulos.csv");
float ang3 = readFloatIEEE(32574 - 30001); salvarNoSD("ang_U3_I3",
ang3, "/angulos.csv");
float angT = readFloatIEEE(32576 - 30001); salvarNoSD("ang_total",
angT, "/angulos.csv");
float ang12 = readFloatIEEE(32578 - 30001); salvarNoSD("ang_U1_U2",
ang12, "/angulos.csv");
float ang23 = readFloatIEEE(32580 - 30001); salvarNoSD("ang_U2_U3",
ang23, "/angulos.csv");
float ang31 = readFloatIEEE(32582 - 30001); salvarNoSD("ang_U3_U1",
ang31, "/angulos.csv");

float freq = readFloatIEEE(32584 - 30001); salvarNoSD("Frequência",
freq, "/frequencia.csv");

float thd_i1 = readFloatIEEE(32588 - 30001); salvarNoSD("THD_I1",
thd_i1, "/thd_correntes.csv");
float thd_i2 = readFloatIEEE(32590 - 30001); salvarNoSD("THD_I2",
thd_i2, "/thd_correntes.csv");
float thd_i3 = readFloatIEEE(32592 - 30001); salvarNoSD("THD_I3",
thd_i3, "/thd_correntes.csv");
float thd_u1 = readFloatIEEE(32594 - 30001); salvarNoSD("THD_U1",
thd_u1, "/thd_tensoes.csv");
float thd_u2 = readFloatIEEE(32596 - 30001); salvarNoSD("THD_U2",
thd_u2, "/thd_tensoes.csv");
float thd_u3 = readFloatIEEE(32598 - 30001); salvarNoSD("THD_U3",
thd_u3, "/thd_tensoes.csv");

float runtime = readFloatIEEE(32480 - 30001); salvarNoSD("Runtime",
runtime, "/runtime.csv");
float kwh1 = readFloatIEEE(32638 - 30001); salvarNoSD("Kwh1", kwh1,
"/energia.csv");
float kwh2 = readFloatIEEE(32640 - 30001); salvarNoSD("Kwh2", kwh2,
"/energia.csv");
float kvarh3 = readFloatIEEE(32642 - 30001); salvarNoSD("Kvarh3",
kvarh3, "/energia.csv");
float kvarh4 = readFloatIEEE(32644 - 30001); salvarNoSD("Kvarh4",
kvarh4, "/energia.csv");

// === ENVIO MQTT ===
if (client.connected()) {
    enviarParte1();
    delay(100);
}
```

```

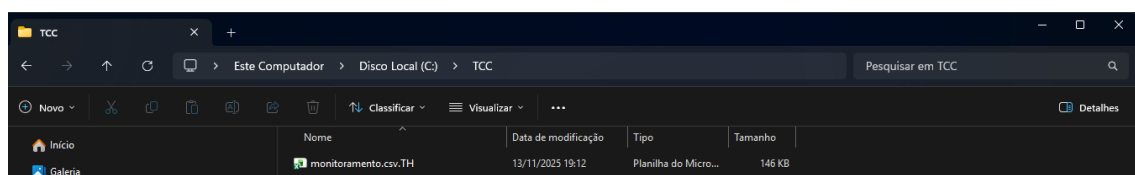
enviarParte2();
Serial.println("2 PARTES ENVIADAS!");
} else {
Serial.println("MQTT OFF → SD OK");
}
}
}

```

Os dados enviados via rede são adicionados a um arquivo chamado “monitoramento.csv” esta no seguinte endereço:

Este Computador > Disco Local (C:) > TCC

Neste mesmo endereço vai estar o arquivo csv no qual usamos para monitoramento das informações , com tabela e gráficos:



Salvamento Automático monitoramento.csv.TH • Salvo neste PC

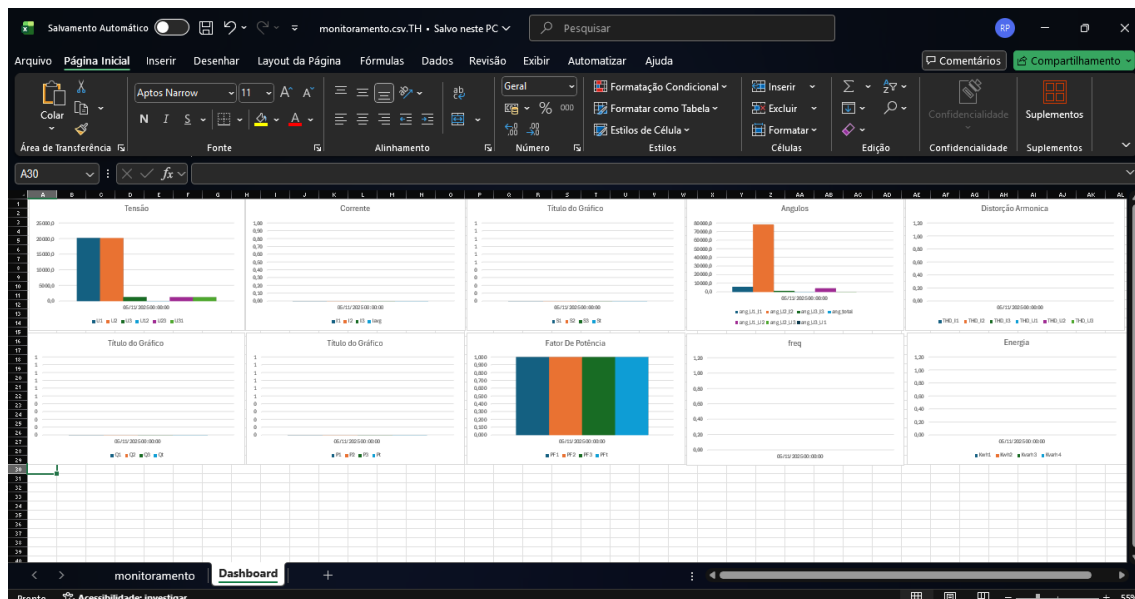
Arquivo Página Inicial Inserir Desenhar Layout da Página Fórmulas Dados Revisão Exibir Automatizar Ajuda Design da Tabela Consulta Comentários Compartilhamento

Área de Transferência Fonte Alinhamento Número Estilos Células Edição Confidencialidade Suplementos

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	Timestamp	U1	U2	U3	U12	U23	U31	I1	I2	I3	Iavg	P1	P2	P3	Pt	Q1	Q2
2	1761.0	175.0	1328.0	0.0	1255.0	1256.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
3	17605.0	175.0	132.0	0.0	1255.0	1259.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
4	195.0	196.0	1328.0	0.0	126.0	126.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
5	17505.0	17725.0	1326.0	0.0	125.0	1254.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
6	1775.0	178.0	1314.0	0.0	1255.0	1258.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
7	1812.0	18225.0	1318.0	0.0	1255.0	1259.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
8	17995.0	179.0	1312.0	0.0	1255.0	1255.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
9	18885.0	189.0	1312.0	0.0	125.0	1252.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
10	18505.0	186.0	1318.0	0.0	125.0	1252.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
11	175.0	176.0	1318.0	0.0	125.0	1251.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
12	19105.0	19.0	131.0	0.0	125.0	1251.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
13	19995.0	20025.0	131.0	0.0	125.0	1252.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
14	19735.0	197.0	1316.0	0.0	125.0	1254.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
15	19735.0	19825.0	1314.0	0.0	125.0	1252.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
16	195.0	19725.0	1326.0	0.0	1255.0	1255.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
17	1803.0	181.0	1318.0	0.0	125.0	1251.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
18	1776.0	17925.0	1312.0	0.0	125.0	1251.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
19	18755.0	188.0	1314.0	0.0	1255.0	1256.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
20	18985.0	19025.0	131.0	0.0	124.0	1243.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
21	1909.0	191.0	1318.0	0.0	1245.0	1245.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
22	1909.0	19.0	131.0	0.0	125.0	1251.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
23	18855.0	189.0	1314.0	0.0	125.0	1251.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0
24	18995.0	189.0	1312.0	0.0	125.0	1252.0	0.00	0.00	0.00	0.00	0.00	0	0	0	0	0	0

monitoramento Dashboard

Pronto Acessibilidade: investigar



Após a energização da placa com o devido código carregado ao esp , e com o “mosquitto” e “Nod-Red” rodando , as informações de grandezas elétrica devem ser atualizadas a cada 10 segundos no excel , a cada varredura do código no cartão SD .